

COS Graph Engine v2.1+ Roadmap

v2.0.0 completado: 20 fases, 68 tickets, 1145 tests, 0 failures

<https://cos-graph-engine.higgsfield.app>

Zero-dep rule: solo Stripe, SendGrid, LangChain, Algolia

Contents

1. Resumen Ejecutivo
2. Fase 1 — Performance Foundations
3. Fase 2 — WASM Acceleration
4. Fase 3 — Telemetry & Observability
5. Fase 4 — Web Visualization
6. Fase 5 — Deployment & Operations
7. Fase 6 — Ecosystem & DX
8. Gantt Chart
9. Dependency Graph
10. Milestones
11. Metricas Clave
12. Estado de Ejecucion

Resumen Ejecutivo

COS Graph Engine v2.1+ transforma el motor de grafos de 20 niveles en un sistema production-grade: rendimiento 2x via CSR + WASM, observabilidad integrada, visualizacion web en tiempo real, despliegue Kubernetes-ready, y un ecosistema de paquetes publicables.

Fase	Nombre	Tickets	Dep	Esfuerzo
1	Performance Foundations	3	—	3-4d
2	WASM Acceleration	3	Fase 1	4-5d
3	Telemetry & Observability	3	Fase 1	2-3d
4	Web Visualization	3	Fase 1	3-4d
5	Deployment & Operations	3	Fases 1-4	4-5d
6	Ecosystem & DX	3	Fases 1-5	3-4d

Total: 18 tickets, ~20-25 dias

Fase 1 — Performance Foundations

Objetivo: Reemplazar Map<string, string[]> por CSR (Compressed Sparse Row) + bidirectional pruning. 2x speedup en BFS/DFS/shortest path.

Dependencias: Ninguna | Duracion: 3-4 dias

T-1.1 — CSR Storage (COMPLETADO)

Archivos: packages/graph/src/csr.ts, scripts/test-csr.ts (77 tests, 0 failures)

Clases: CSRGraph<N,E> con addNode/removeNode, addEdge/removeEdge, neighbors(), bfs(), dfs(), bidirectionalBFS(), degree(), toJSON/fromJSON, clear(). CompressedAdjacency como drop-in replacement para Map<string, string[]>.

Estructura CSR: indices[] flat array de targets, indptr[] row pointers, nodeIds[] row -> node ID. Algoritmos: BFS, bidirectional BFS (meet-in-the-middle), DFS iterativo, reverse neighbors scan. Performance: rebuild O(N+E), neighbors O(degree), BFS O(N+E).

T-1.2 — Bidirectional Pruning

Estrategias de poda configurables para BFS, DFS y shortest path. Reduccion del espacio de busqueda hasta 60%.

#	Strategy	Mecanismo	Reduccion
1	MaxDepthPruning	Corta en depth >= maxDepth	—
2	VisitedPruning	Skip si ya visitado	30-50%
3	TargetDirectionPruning	Target en subarbol inverso	20-40%
4	CostBoundPruning	Costo > mejor conocido	15-30%
5	BeamPruning	Top-K candidatos por nivel	40-60%
6	LandmarkPruning	Distancia via landmarks	35-50%
7	EarlyExitPruning	Corta al encontrar target	50-80%

Sub-tickets: 1.2a Core Pruning Engine (4-5h), 1.2b Estrategias Built-in (4-5h), 1.2c Integracion CSRGraph (2-3h), 1.2d Integracion Niveles L0-L19 (3-4h), 1.2e Tests (3-4h). Total: 68 tests.

T-1.3 — Benchmark Suite

Benchmarks reproducibles: BFS chain (10K nodos), BFS grid (100x100), BFS social (5K, grado 20), shortest path, pruning comparison, memory profile (1K-100K). Target: 2x speedup, < 50% memoria en grafos sparse.

Fase 2 — WASM Acceleration

Objetivo: Compilar core math a WebAssembly real. Hot paths en WASM.

Dependencias: Fase 1 | Duracion: 4-5 dias

T-2.1 — AssemblyScript Pipeline

Pipeline de compilacion AS -> WASM. Modulos: csr.wasm (BFS), pagerank.wasm (power iteration), shortest.wasm (bidirectional Dijkstra), centrality.wasm (betweenness).

T-2.2 — WASM Loader

WASMModule interface con bfs, pageRank, shortestPath. WASMLoader.load() con fallback automatico a JS puro si WebAssembly no esta soportado.

T-2.3 — WASM Benchmarks

Benchmarks: BFS 10K (2.5-3x), PageRank 5K (3-4x), Dijkstra 10K (2.5-3x), Betweenness 1K (3-4x).

Fase 3 — Telemetry & Observability

Objetivo: Tracing por hop, profiling por operacion, exportacion OpenTelemetry.

Dependencias: Fase 1 | Duracion: 2-3 dias

T-3.1 — Per-Hop Tracing

TraceSession con hops, bidirectional flag, pruned count. NoopTraceSession para zero overhead cuando no se usa.

T-3.2 — Profiling Hooks

Profiler con start/snapshot/summary. Exportacion Prometheus: cos_graph_bfs_duration_ms, cos_graph_memory_bytes, cos_graph_operations_total.

T-3.3 — Telemetry Plugin

Plugin @cos/telemetry con onBeforeOperation/onAfterOperation/onError. Exportadores: Console, Prometheus (/metrics), OpenTelemetry (OTLP).

Fase 4 — Web Visualization

Objetivo: SVG (documentos), Canvas (10K+ nodos/30fps), Web Component (drop-in).

Dependencias: Fase 1 | Duracion: 3-4 dias

T-4.1 — SVG Renderer

SVGGraphRenderer.render() con force-directed layout zero-dep: repulsion Coulomb, atraccion Hooke, Barnes-Hut optimizacion.

T-4.2 — Canvas Renderer

CanvasGraphRenderer con quadtree culling, zoom/pan, click/hover/drag. 10K+ nodos a 30fps.

T-4.3 — Web Component

<cos-graph> custom element con graphData, layout (force/tree/radial), theme, focusNode(), highlightPath(), exportSVG(), exportPNG().

Fase 5 — Deployment & Operations

Objetivo: K8s operator, single-binary SEA, CI/CD automatizado.

Dependencias: Fases 1-4 | Duracion: 4-5 dias

T-5.1 — Kubernetes Operator

CRD GraphEngine con levels, size, persistence, replication, monitoring. Controller con reconciliation loop, auto-scaling, backup CronJob.

T-5.2 — Single-Binary SEA

cos-engine serve/repl/import/export/benchmark/status. Binarios: Linux (28MB), macOS (24MB), Windows (30MB).

T-5.3 — CI/CD Pipeline

Workflows: test.yml, wasm.yml, build.yml, publish.yml, deploy.yml. Quality gates: 100% tests, >85% coverage, WASM <500KB, speedup >1.5x.

Fase 6 — Ecosystem & DX

Objetivo: Monorepo publicado como paquetes npm, documentacion interactiva, release 1-command.

Dependencias: Fases 1-5 | Duracion: 3-4 dias

T-6.1 — Monorepo Split

7 paquetes: @cos/core (0 deps), @cos/wasm, @cos/telemetry, @cos/vis, @cos/cli, @cos/api, @cos/ml.

T-6.2 — Docs Site Interactivo

VitePress con 7 secciones: Introduccion, Quickstart, CSR Guide, WASM Guide, API Reference, Visualization, Deployment.
WASM Playground en navegador.

T-6.3 — Release Automation

1 comando: npm run release -- --type minor. Bump version, CHANGELOG, tag, GitHub Release, npm publish, Docker image.

Gantt Chart

18 tickets en ~15-20 dias habiles. Fase 1 y 4 arrancan en paralelo (dependen solo de T-1.1). Fase 2 espera a Fase 1 completa. Fase 5 espera Fase 2 + 3 + 4. Fase 6 cierra el ciclo.

Fase	Ticket	Duracion	Inicio	Dep
1	T-1.1 CSR Storage	1d	Dia 1	—
1	T-1.2 Pruning	2d	Dia 2	T-1.1
1	T-1.3 Benchmarks	1d	Dia 4	T-1.2
2	T-2.1 AS Pipeline	2d	Dia 5	Fase 1
2	T-2.2 WASM Loader	1d	Dia 7	T-2.1
2	T-2.3 WASM Bench	1d	Dia 8	T-2.2
3	T-3.1 Tracing	1d	Dia 3	T-1.2
3	T-3.2 Profiling	1d	Dia 4	T-3.1
3	T-3.3 Plugin	1d	Dia 5	T-3.2
4	T-4.1 SVG	1d	Dia 2	T-1.1
4	T-4.2 Canvas	1d	Dia 3	T-4.1
4	T-4.3 Web Comp	1d	Dia 4	T-4.2

5	T-5.1 K8s	2d	Dia 9	T-2.2+T-3.3
5	T-5.2 SEA	1d	Dia 9	T-2.2+T-4.3
5	T-5.3 CI/CD	1d	Dia 5	T-1.3
6	T-6.1 Split	1d	Dia 11	Fase 5
6	T-6.2 Docs	1d	Dia 12	T-6.1
6	T-6.3 Release	1d	Dia 12	T-6.1

Dependency Graph

Fase 1 -> Fase 2,3,4. Fase 2,3,4 -> Fase 5. Fase 5 -> Fase 6.

T-1.1 -> T-1.2 -> T-1.3 -> Fase 2. T-1.2 -> Fase 3. T-1.1 -> Fase 4.

T-2.2 + T-3.3 + T-4.3 -> Fase 5. T-1.3 -> T-5.3. T-2.2 + T-4.3 -> T-5.2.

Milestones

M1 — Performance

CSR + pruning + benchmarks. 2x speedup, 137 tests nuevos.

M2 — WASM Ready

Hot paths en WASM, fallback JS. 3x PageRank, 2.5x BFS.

M3 — Observable

Tracing, profiling, Prometheus. <1% overhead sin tracing.

M4 — Visual

SVG + Canvas 10K/30fps + Web Component.

M5 — Deployable

K8s operator, SEA binary, CI/CD. Deploy en <5 min.

M6 — Launch

7 npm packages, docs site, release automation. 1500+ tests.

Metricas Clave

Metrica	v2.0	v2.1 Target	Mejora
BFS 10K nodos	3.5ms	1.8ms	2x
Memoria 100K sparse	42MB	<10MB	4x
PageRank 5K	45ms	12ms	3.75x
Shortest path 10K	12ms	4ms	3x
Tests totales	1068	1500+	+40%
Paquetes npm	0	7	—
Deploy time	manual	5 min	—
Visualizacion	—	10K/30fps	—
WASM support	simulado	real	—
K8s support	—	CRD + operator	—
Docker image	—	<50MB	—

Estado de Ejecucion

Fase 1 en progreso (28%). T-1.1 CSR Storage completado (77 tests, 0 failures). T-1.2 Pruning pendiente. T-1.3 Benchmarks pendiente. Fases 2-6 pendientes.